

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №1.

Тема: «Графы»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

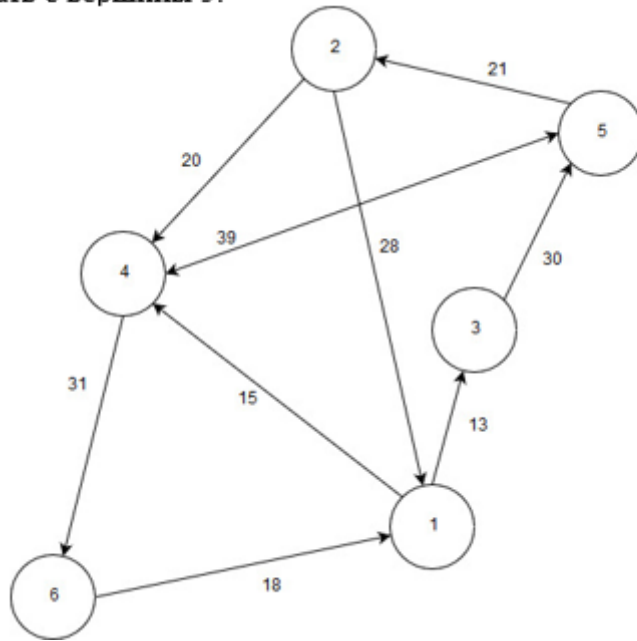
Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

Матрица смежности:



Анализ задачи:

Проект разделен на три логических уровня (паттерн Model-View-Controller):

Model (Graph класс): Отвечает только за логику. Хранит данные (списки смежности) и выполняет математические расчеты алгоритмов. Не знает ничего о кнопках и окнах.

View (GraphWidget): Отвечает за отрисовку. Использует QPainter для рисования линий и эллипсов. Принимает команды о цветах (какую вершину подсветить красным, а какую зеленым).

Controller (MainWindow): Связывает UI и Модель. Обрабатывает нажатия кнопок, запускает таймер анимации (QTimer) и обновляет статус.

Код:

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(GraphVisualizer)

set(CMAKE_CXX_STANDARD 17)
```

```
# Включаем поддержку MOC, RCC и UIC для Qt
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)
set(CMAKE_AUTOUIC ON)

set(CMAKE_PREFIX_PATH "C:/Qt/6.11.1/mingw_64")

# Находим пакет Qt6 (нам нужен только Widgets для
начала)
find_package(Qt6 COMPONENTS Widgets REQUIRED)

# Создаем исполняемый файл
add_executable(GraphVisualizer
    main.cpp
    mainwindow.cpp
    graphwidget.cpp
    graph.cpp
    graph.cpp
    graph.h
    graphwidget.cpp
    graphwidget.h
    mainwindow.cpp
    mainwindow.h
)

# Линкуем библиотеки Qt
target_link_libraries(GraphVisualizer Qt6::Widgets)
```

graph.cpp

```
#include "graph.h"
#include <queue>
#include <stack>

void Graph::addVertex(int id, QPointF pos) {
    vertices.insert(id);
    if (pos.isNull()) {
        positions[id] = QPointF(0, 0);
    } else {
        positions[id] = pos;
    }
}

void Graph::removeVertex(int id) {
    vertices.erase(id);
    adjacencyList.erase(id);
    positions.erase(id);

    // Удаляем все рёбра, ведущие к этой вершине
    for (auto& pair : adjacencyList) {
        pair.second.erase(
            std::remove_if(pair.second.begin(),
pair.second.end(),
                [id](const Edge& e) { return e.to ==
id; })),
        pair.second.end()
    );
    }
}
```

```

void Graph::addEdge(int from, int to, double weight) {
    vertices.insert(from);
    vertices.insert(to);
    adjacencyList[from].emplace_back(to, weight);
}

void Graph::removeEdge(int from, int to) {
    if (adjacencyList.count(from)) {
        adjacencyList[from].erase(
            std::remove_if(adjacencyList[from].begin(),
adjacencyList[from].end(),
                [to](const Edge& e) { return e.to ==
to; })),
        adjacencyList[from].end()
    );
    }
}

std::vector<Edge> Graph::getEdges(int vertex) const {
    auto it = adjacencyList.find(vertex);
    if (it != adjacencyList.end()) {
        return it->second;
    }
    return {};
}

QPointF Graph::getPosition(int id) const {
    auto it = positions.find(id);
    if (it != positions.end()) {

```

```

        return it->second;
    }
    return QPointF();
}

void Graph::setPosition(int id, QPointF pos) {
    positions[id] = pos;
}

// BFS - обход в ширину
std::vector<int> Graph::bfs(int start) {
    std::vector<int> result;
    std::set<int> visited;
    std::queue<int> q;

    q.push(start);
    visited.insert(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        result.push_back(v);

        for (const auto& edge : getEdges(v)) {
            if (visited.find(edge.to) == visited.end())
            {
                visited.insert(edge.to);
                q.push(edge.to);
            }
        }
    }
}

```

```

    }

    }

    return result;
}

// DFS - обход в глубину
std::vector<int> Graph::dfs(int start) {
    std::vector<int> result;
    std::set<int> visited;
    std::stack<int> s;

    s.push(start);

    while (!s.empty()) {
        int v = s.top();
        s.pop();

        if (visited.find(v) == visited.end()) {
            visited.insert(v);
            result.push_back(v);

            for (const auto& edge : getEdges(v)) {
                if (visited.find(edge.to) ==
visited.end()) {
                    s.push(edge.to);
                }
            }
        }
    }
}

```

```

        return result;
    }

    // Алгоритм Дейкстры
    std::map<int, double> Graph::dijkstra(int start) {
        std::map<int, double> dist;
        std::set<std::pair<double, int>> q;

        for (int v : vertices) {
            dist[v] =
std::numeric_limits<double>::infinity();
        }
        dist[start] = 0;
        q.insert({0, start});

        while (!q.empty()) {
            int v = q.begin()->second;
            q.erase(q.begin());

            for (const auto& edge : getEdges(v)) {
                double alt = dist[v] + edge.weight;
                if (alt < dist[edge.to]) {
                    q.erase({dist[edge.to], edge.to});
                    dist[edge.to] = alt;
                    q.insert({alt, edge.to});
                }
            }
        }

        return dist;
    }

```



```

}

// Алгоритм Флойда-Уоршелла
std::map<int, std::map<int, double>> Graph::floyd() {
    std::map<int, std::map<int, double>> dist;

    // Инициализация
    for (int u : vertices) {
        for (int v : vertices) {
            dist[u][v] = (u == v) ? 0 :
std::numeric_limits<double>::infinity();
        }
    }

    for (int u : vertices) {
        for (const auto& edge : getEdges(u)) {
            dist[u][edge.to] = edge.weight;
        }
    }

    // Основной алгоритм
    for (int k : vertices) {
        for (int i : vertices) {
            for (int j : vertices) {
                if (dist[i][k] + dist[k][j] <
dist[i][j]) {
                    dist[i][j] = dist[i][k] +
dist[k][j];
                }
            }
        }
    }
}

```

```

        }

    }

    return dist;
}

```

graph.h

```

#ifndef GRAPH_H
#define GRAPH_H

#include <vector>
#include <map>
#include <set>
#include <limits>
#include <QPointF>

struct Edge {
    int to;
    double weight;
    Edge(int t, double w) : to(t), weight(w) {}
};

class Graph {
private:
    std::map<int, std::vector<Edge>> adjacencyList;
    std::map<int, QPointF> positions; // Позиции вершин
    для отрисовки
    std::set<int> vertices;

public:

```

```

// Основные операции

void addVertex(int id, QPointF pos = QPointF());
void removeVertex(int id);

void addEdge(int from, int to, double weight =
1.0);

void removeEdge(int from, int to);


// Получение данных

std::set<int> getVertices() const { return
vertices; }

std::vector<Edge> getEdges(int vertex) const;
QPointF getPosition(int id) const;
void setPosition(int id, QPointF pos);


// Алгоритмы

std::vector<int> bfs(int start);
// Обход в ширину

std::vector<int> dfs(int start);
// Обход в глубину

std::map<int, double> dijkstra(int start);
// Дейкстра

std::map<int, std::map<int, double>> floyd();
// Флойд
};

#endif // GRAPH_H

```

graphwidget.cpp

```

#include "graphwidget.h"
#include <QPainterPath>

```

```
GraphWidget::GraphWidget(QWidget* parent)
    : QWidget(parent), graph(nullptr),
    selectedVertex(-1), highlightVertex(-1),
    dragging(false) {
    setMinimumSize(600, 400);
    setMouseTracking(true);
}

void GraphWidget::setGraph(Graph* g) {
    graph = g;
    update();
}

void GraphWidget::setHighlightVertex(int id) {
    highlightVertex = id;
    update();
}

void GraphWidget::setVisitedVertices(const
std::vector<int>& vertices) {
    visitedVertices = vertices;
    update();
}

void GraphWidget::paintEvent(QPaintEvent* event) {
    QWidget::paintEvent(event);

    if (!graph) return;

    QPainter painter(this);
```

```

painter.setRenderHint(QPainter::Antialiasing);

// Рисуем рёбра
for (int v : graph->getVertices()) {
    for (const auto& edge : graph->getEdges(v)) {
        drawEdge(painter, v, edge.to, edge.weight);
    }
}

// Рисуем вершины
for (int v : graph->getVertices()) {
    bool selected = (v == selectedVertex);
    bool highlighted = (v == highlightVertex);
    bool visited =
std::find(visitedVertices.begin(),
visitedVertices.end(), v) != visitedVertices.end();
    drawVertex(painter, v, graph->getPosition(v),
selected, highlighted, visited);
}
}

void GraphWidget::drawVertex(QPainter& painter, int
id, QPointF pos, bool selected, bool highlighted, bool
visited) {
    int radius = selected ? 22 : (highlighted ? 25 :
18);

    QColor color;

    if (highlighted) {
        color = QColor(255, 87, 34); //
Оранжево-красный (текущий шаг)

```

```

    } else if (visited) {
        color = QColor(76, 175, 80); // Зелёный
(посещён)
    } else {
        color = QColor(33, 150, 243); // Синий (не
посещён)
    }

    // Тень для выделенной вершины
    if (highlighted) {
        painter.setBrush(QColor(255, 235, 59)); //
Жёлтая тень
        painter.setPen(QPen(Qt::black, 3));
        painter.drawEllipse(pos, radius + 3, radius +
3);
    }

    // Круг
    painter.setBrush(color);
    painter.setPen(QPen(Qt::black, 2));
    painter.drawEllipse(pos, radius, radius);

    // Текст с номером
    painter.setPen(Qt::white);
    painter.setFont(QFont("Arial", 12, QFont::Bold));
    painter.drawText(QRectF(pos.x()-radius,
pos.y()-radius, radius*2, radius*2),
                    Qt::AlignCenter,
QString::number(id));
}

```

```

void GraphWidget::drawEdge(QPainter& painter, int
from, int to, double weight) {
    QPointF p1 = graph->getPosition(from);
    QPointF p2 = graph->getPosition(to);

    // Проверяем, является ли ребро частью текущего
шага

    bool isHighlighted = (from == highlightVertex || to
== highlightVertex);

    // Рисуем линию
    painter.setPen(QPen(isHighlighted ? QColor(255, 87,
34) : Qt::gray,
                        isHighlighted ? 4 : 2));
    painter.drawLine(p1, p2);

    // Рисуем вес (посередине)
    QPointF mid = (p1 + p2) / 2;
    painter.setPen(isHighlighted ? QColor(255, 87, 34)
: Qt::black);
    painter.setFont(QFont("Arial", 10, QFont::Bold));
    painter.drawText(mid, QString::number(weight));
}

void GraphWidget::mousePressEvent(QMouseEvent* event)
{
    if (!graph) return;

    QPointF pos = event->pos();

    // Проверяем клик по вершине

```

```

        for (int v : graph->getVertices()) {
            QPointF vPos = graph->getPosition(v);
            if (QLineF(pos, vPos).length() < 20) {
                selectedVertex = v;
                dragging = true;
                lastMousePos = pos;
                update();
                return;
            }
        }
    }

void GraphWidget::mouseMoveEvent(QMouseEvent* event) {
    if (dragging && selectedVertex != -1 && graph) {
        QPointF newPos =
graph->getPosition(selectedVertex) + (event->pos() -
lastMousePos);

        graph->setPosition(selectedVertex, newPos);
        lastMousePos = event->pos();
        update();
    }
}

void GraphWidget::mouseReleaseEvent(QMouseEvent*
event) {
    dragging = false;
}

```

graphwidget.h

```

#ifndef GRAPHWIDGET_H

```



```
#define GRAPHWIDGET_H

#include <QWidget>
#include <QPainter>
#include <QMouseEvent>
#include "graph.h"

class GraphWidget : public QWidget {
    Q_OBJECT

private:
    Graph* graph;
    int selectedVertex;
    int highlightVertex; // Подсвеченная вершина
                          (текущий шаг)
    std::vector<int> visitedVertices; // Посещённые
    вершины
    bool dragging;
    QPointF lastMousePos;

public:
    explicit GraphWidget(QWidget* parent = nullptr);
    void setGraph(Graph* g);
    void setHighlightVertex(int id);
    void setVisitedVertices(const std::vector<int>&
vertices);

protected:
    void paintEvent(QPaintEvent* event) override;
    void mousePressEvent(QMouseEvent* event) override;
```

```

        void mouseMoveEvent(QMouseEvent* event) override;

        void mouseReleaseEvent(QMouseEvent* event)
override;

private:

        void drawVertex(QPainter& painter, int id, QPointF
pos, bool selected, bool highlighted, bool visited);

        void drawEdge(QPainter& painter, int from, int to,
double weight);

};

#endif // GRAPHWIDGET_H

```

mainwindow.cpp

```

#include "mainwindow.h"

#include <queue>           // Для очереди (BFS)
#include <stack>           // Для стека (DFS)
#include <set>             // Для множеств (чтобы не
посещать вершины дважды)
#include <limits>         // Для бесконечности в Дейкстре

MainWindow::MainWindow(QWidget* parent)
    : QMainWindow(parent), animationTimer(new
QTimer(this)) {

    graph = new Graph();

    setupUI();

    setupInitialGraph();

    connect(animationTimer, &QTimer::timeout, this,
[this]() {

```

```
        // Таймер будет переключаться в зависимости
от алгоритма

    });
}

MainWindow::~MainWindow() {
    delete graph;
}

void MainWindow::setupInitialGraph() {
    // Инициализация графа из картинки
    // Вершины с координатами
    graph->addVertex(1, QPointF(400, 350));
    graph->addVertex(2, QPointF(300, 50));
    graph->addVertex(3, QPointF(450, 200));
    graph->addVertex(4, QPointF(200, 150));
    graph->addVertex(5, QPointF(500, 80));
    graph->addVertex(6, QPointF(150, 300));

    // Рёбра
    graph->addEdge(2, 4, 20);
    graph->addEdge(2, 1, 28);
    graph->addEdge(5, 2, 21);
    graph->addEdge(5, 4, 39);
    graph->addEdge(4, 6, 31);
    graph->addEdge(1, 4, 15);
    graph->addEdge(1, 3, 13);
    graph->addEdge(1, 5, 30);
    graph->addEdge(6, 1, 18);
}
```

```
graphWidget->update();
}

void MainWindow::setupUI() {
    setWindowTitle("Визуализация графов - Пошаговый алгоритм");
    resize(1100, 800);

    QWidget* centralWidget = new QWidget(this);
    setCentralWidget(centralWidget);

    QVBoxLayout* mainLayout = new
    QVBoxLayout(centralWidget);

    // Панель управления
    QHBoxLayout* controllLayout = new QHBoxLayout();

    controllLayout->addWidget(new QLabel("Вершина:"));
    vertexIdEdit = new QLineEdit();
    vertexIdEdit->setPlaceholderText("ID");
    vertexIdEdit->setMaximumWidth(60);
    controllLayout->addWidget(vertexIdEdit);

    QPushButton* addVertexBtn = new
    QPushButton("Добавить вершину");
    connect(addVertexBtn, &QPushButton::clicked, this,
    &MainWindow::addVertex);
    controllLayout->addWidget(addVertexBtn);
}
```

```
QPushButton* removeVertexBtn = new
QPushButton("Удалить вершину");

connect(removeVertexBtn, &QPushButton::clicked,
this, &MainWindow::removeVertex);

controlLayout->addWidget(removeVertexBtn);

controlLayout->addSpacing(20);

controlLayout->addWidget(new QLabel("Ребро:"));
edgeFromEdit = new QLineEdit();
edgeFromEdit->setPlaceholderText("От");
edgeFromEdit->setMaximumWidth(50);
controlLayout->addWidget(edgeFromEdit);

edgeToEdit = new QLineEdit();
edgeToEdit->setPlaceholderText("До");
edgeToEdit->setMaximumWidth(50);
controlLayout->addWidget(edgeToEdit);

weightEdit = new QLineEdit();
weightEdit->setPlaceholderText("Вес");
weightEdit->setMaximumWidth(50);
weightEdit->setText("1");
controlLayout->addWidget(weightEdit);

QPushButton* addEdgeBtn = new QPushButton("Добавить
ребро");

connect(addEdgeBtn, &QPushButton::clicked, this,
&MainWindow::addEdge);

controlLayout->addWidget(addEdgeBtn);
```

```
QPushButton* removeEdgeBtn = new
QPushButton("Удалить ребро");

    connect(removeEdgeBtn, &QPushButton::clicked, this,
&MainWindow::removeEdge);

    controlLayout->addWidget(removeEdgeBtn);

mainLayout->addLayout(controlLayout);

// Панель алгоритмов
QHBoxLayout* algoLayout = new QHBoxLayout();
algoLayout->addWidget(new QLabel("Старт:"));
startVertexEdit = new QLineEdit();
startVertexEdit->setPlaceholderText("ID");
startVertexEdit->setMaximumWidth(50);
startVertexEdit->setText("1");
algoLayout->addWidget(startVertexEdit);

QPushButton* bfsBtn = new QPushButton("BFS
(пошагово)");

    connect(bfsBtn, &QPushButton::clicked, this,
&MainWindow::startBFS);

    bfsBtn->setStyleSheet("background-color: #4CAF50;
color: white; padding: 5px;");

    algoLayout->addWidget(bfsBtn);

QPushButton* dfsBtn = new QPushButton("DFS
(пошагово)");

    connect(dfsBtn, &QPushButton::clicked, this,
&MainWindow::startDFS);
```

```
    dfsBtn->setStyleSheet("background-color: #2196F3;
color: white; padding: 5px;");

    algoLayout->addWidget(dfsBtn);

    QPushButton* dijkstraBtn = new
QPushButton("Дейкстра (пошагово)");

    connect(dijkstraBtn, &QPushButton::clicked, this,
&MainWindow::startDijkstra);

    dijkstraBtn->setStyleSheet("background-color:
#FF9800; color: white; padding: 5px;");

    algoLayout->addWidget(dijkstraBtn);

    QPushButton* floydBtn = new QPushButton("Флойд");

    connect(floydBtn, &QPushButton::clicked, this,
&MainWindow::runFloyd);

    floydBtn->setStyleSheet("background-color: #9C27B0;
color: white; padding: 5px;");

    algoLayout->addWidget(floydBtn);

    mainLayout->addLayout(algoLayout);

    // Индикатор шага

    stepLabel = new QLabel("Готов к работе");

    stepLabel->setStyleSheet("QLabel { font-size: 14px;
font-weight: bold; color: #333; background-color:
#E3F2FD; padding: 8px; border-radius: 5px; }");

    mainLayout->addWidget(stepLabel);

    // Виджет графа

    graphWidget = new GraphWidget();

    graphWidget->setGraph(graph);
```

```
mainLayout->addWidget(graphWidget, 1);

// Результат
resultLabel = new QLabel("Результат: ");
resultLabel->setWordWrap(true);
resultLabel->setStyleSheet("QLabel {
background-color: #f0f0f0; padding: 10px;
border-radius: 5px; font-family: monospace; }");
mainLayout->addWidget(resultLabel);
}

void MainWindow::addVertex() {
    bool ok;
    int id = vertexIdEdit->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(this, "Ошибка", "Введите
корректный ID вершины");
        return;
    }

    QPointF pos(100 + rand() % 400, 100 + rand() %
200);
    graph->addVertex(id, pos);
    graphWidget->update();
    vertexIdEdit->clear();
}

void MainWindow::removeVertex() {
    bool ok;
    int id = vertexIdEdit->text().toInt(&ok);
```



```
        if (!ok) {
            QMessageBox::warning(this, "Ошибка", "Введите
корректный ID вершины");
            return;
        }

        graph->removeVertex(id);
        graphWidget->update();
        vertexIdEdit->clear();
    }

void MainWindow::addEdge() {
    bool ok1, ok2, ok3;
    int from = edgeFromEdit->text().toInt(&ok1);
    int to = edgeToEdit->text().toInt(&ok2);
    double weight = weightEdit->text().toDouble(&ok3);

    if (!ok1 || !ok2 || !ok3) {
        QMessageBox::warning(this, "Ошибка", "Проверьте
введенные данные");
        return;
    }

    graph->addEdge(from, to, weight);
    graphWidget->update();
}

void MainWindow::removeEdge() {
    bool ok1, ok2;
    int from = edgeFromEdit->text().toInt(&ok1);
```

```
int to = edgeToEdit->text().toInt(&ok2);

if (!ok1 || !ok2) {
    QMessageBox::warning(this, "Ошибка", "Проверьте введенные данные");
    return;
}

graph->removeEdge(from, to);
graphWidget->update();
}

// ===== BFS =====

void MainWindow::startBFS() {
    bool ok;
    int start = startVertexEdit->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(this, "Ошибка", "Введите стартовую вершину");
        return;
    }

    // Подготовка данных для анимации
    bfsVisited.clear();
    bfsQueue.clear();
    bfsCurrentVertex = -1;

    std::set<int> visited;
    std::queue<int> q;
```

```

q.push(start);
visited.insert(start);
bfsQueue.push_back(start);

// Сохраняем порядок обхода для анимации
while (!q.empty()) {
    int v = q.front();
    q.pop();

    for (const auto& edge : graph->getEdges(v)) {
        if (visited.find(edge.to) == visited.end())
        {
            visited.insert(edge.to);
            q.push(edge.to);
            bfsQueue.push_back(edge.to);
        }
    }
}

bfsVisited = std::vector<int>(visited.begin(),
visited.end());
currentStep = 0;

stepLabel->setText("Шаг 1: Начинаем BFS с вершины "
+ QString::number(start));
graphWidget->setHighlightVertex(start);
graphWidget->setVisitedVertices({});
graphWidget->update();

// Запускаем таймер для анимации

```

```

        disconnect(animationTimer, nullptr, nullptr,
nullptr);

        connect(animationTimer, &QTimer::timeout, this,
&MainWindow::animateBFSStep);

        animationTimer->start(1000); // 1 секунда между
шагами
    }

void MainWindow::animateBFSStep() {
    if (currentStep >= bfsQueue.size()) {
        animationTimer->stop();
        stepLabel->setText("BFS завершён!");
        resultLabel->setText("Порядок обхода: ");
        for (int v : bfsQueue) {
            resultLabel->setText(resultLabel->text() +
QString::number(v) + " ");
        }
        graphWidget->setHighlightVertex(-1);
        return;
    }

    int vertex = bfsQueue[currentStep];
    currentStep++;

    stepLabel->setText("Шаг " +
QString::number(currentStep) + ": Посещаем вершину " +
QString::number(vertex));

    graphWidget->setHighlightVertex(vertex);

    std::vector<int> visited = bfsVisited;
    visited.resize(currentStep);

```

```

graphWidget->setVisitedVertices(visited);
graphWidget->update();
}

// ===== DFS =====

void MainWindow::startDFS() {
    bool ok;
    int start = startVertexEdit->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(this, "Ошибка", "Введите
стартовую вершину");
        return;
    }

    dfsVisited.clear();
    dfsStack.clear();
    dfsCurrentVertex = -1;

    std::set<int> visited;
    std::stack<int> s;
    std::vector<int> order;

    s.push(start);

    while (!s.empty()) {
        int v = s.top();
        s.pop();

        if (visited.find(v) == visited.end()) {

```

```

        visited.insert(v);
        order.push_back(v);

        for (const auto& edge : graph->getEdges(v))
        {
            if (visited.find(edge.to) ==
visited.end()) {
                s.push(edge.to);
            }
        }
    }
}

dfsVisited = order;
currentStep = 0;

    stepLabel->setText("Шаг 1: Начинаем DFS с вершины "
+ QString::number(start));
    graphWidget->setHighlightVertex(start);
    graphWidget->setVisitedVertices({});
    graphWidget->update();

    disconnect(animationTimer, nullptr, nullptr,
nullptr);

    connect(animationTimer, &QTimer::timeout, this,
&MainWindow::animateDFSStep);

    animationTimer->start(1000);
}

void MainWindow::animateDFSStep() {

```

```

        if (currentStep >= dfsVisited.size()) {
            animationTimer->stop();
            stepLabel->setText("DFS завершён!");
            resultLabel->setText("Порядок обхода: ");
            for (int v : dfsVisited) {
                resultLabel->setText(resultLabel->text() +
QString::number(v) + " ");
            }
            graphWidget->setHighlightVertex(-1);
            return;
        }

        int vertex = dfsVisited[currentStep];
        currentStep++;

        stepLabel->setText("Шаг " +
QString::number(currentStep) + ": Посещаем вершину " +
QString::number(vertex));

        graphWidget->setHighlightVertex(vertex);

        std::vector<int> visited(dfsVisited.begin(),
dfsVisited.begin() + currentStep);

        graphWidget->setVisitedVertices(visited);
        graphWidget->update();
    }

// ===== Dijkstra =====

void MainWindow::startDijkstra() {
    bool ok;

    int start = startVertexEdit->text().toInt(&ok);

```

```
    if (!ok) {  
        QMessageBox::warning(this, "Ошибка", "Введите  
стартовую вершину");  
        return;  
    }  
  
    dijkstraVisited.clear();  
    dijkstraCurrentVertex = -1;  
    dijkstraDist.clear();  
  
    std::set<std::pair<double, int>> q;  
    std::map<int, double> dist;  
    std::vector<int> order;  
  
    for (int v : graph->getVertices()) {  
        dist[v] =  
std::numeric_limits<double>::infinity();  
    }  
    dist[start] = 0;  
    q.insert({0, start});  
  
    while (!q.empty()) {  
        int v = q.begin()->second;  
        q.erase(q.begin());  
        order.push_back(v);  
  
        for (const auto& edge : graph->getEdges(v)) {  
            double alt = dist[v] + edge.weight;  
            if (alt < dist[edge.to]) {  
                q.erase({dist[edge.to], edge.to});  
                q.insert({alt, edge.to});  
            }  
        }  
    }  
    return order;
```



```

        dist[edge.to] = alt;
        q.insert({alt, edge.to});
    }
}

dijkstraVisited = order;
dijkstraDist = dist;
currentStep = 0;

stepLabel->setText("Шаг 1: Начинаем алгоритм Дейкстры с вершины " + QString::number(start));
graphWidget->setHighlightVertex(start);
graphWidget->setVisitedVertices({});
graphWidget->update();

disconnect(animationTimer, nullptr, nullptr, nullptr);

connect(animationTimer, &QTimer::timeout, this, &MainWindow::animateDijkstraStep);

animationTimer->start(1200);
}

void MainWindow::animateDijkstraStep() {
    if (currentStep >= dijkstraVisited.size()) {
        animationTimer->stop();

        stepLabel->setText("Алгоритм Дейкстры завершён!");

        resultLabel->setText("Кратчайшие расстояния от вершины " + startVertexEdit->text() + ":\n");
    }
}

```

```

        for (const auto& pair : dijkstraDist) {
            resultLabel->setText(resultLabel->text() +
                                QString("  Вершина %1:
%2\n").arg(pair.first).arg(pair.second));
        }

        graphWidget->setHighlightVertex(-1);

        return;
    }

    int vertex = dijkstraVisited[currentStep];
    currentStep++;

    stepLabel->setText("Шаг " +
                      QString::number(currentStep) +
                      ": Обрабатываем вершину " +
                      QString::number(vertex) +
                      " (расстояние = " +
                      QString::number(dijkstraDist[vertex]) + ")");
    graphWidget->setHighlightVertex(vertex);

    std::vector<int> visited(dijkstraVisited.begin(),
                           dijkstraVisited.begin() + currentStep);
    graphWidget->setVisitedVertices(visited);
    graphWidget->update();
}

// ===== Floyd =====

void MainWindow::runFloyd() {
    auto result = graph->floyd();

    QString text = "Алгоритм Флойда-Уоршелла
(кратчайшие пути между всеми парами):\n\n";

```

```

        for (const auto& row : result) {
            for (const auto& col : row.second) {
                if (col.second <
std::numeric_limits<double>::infinity()) {
                    text += QString("%1->%2: %3
").arg(row.first).arg(col.first).arg(col.second);
                } else {
                    text += QString("%1->%2: ∞
").arg(row.first).arg(col.first);
                }
            }
            text += "\n";
        }

        resultLabel->setText(text);

        stepLabel->setText("Алгоритм Флойда выполнен (без
анимации - слишком много шагов)");
    }

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QPushButton>
#include <QLineEdit>
#include <QLabel>

```

```
#include <QMessageBox>
#include <QTimer>
#include "graphwidget.h"
#include "graph.h"

class MainWindow : public QMainWindow {
    Q_OBJECT

private:
    Graph* graph;
    GraphWidget* graphWidget;

    QLineEdit* vertexIdEdit;
    QLineEdit* edgeFromEdit;
    QLineEdit* edgeToEdit;
    QLineEdit* weightEdit;
    QLineEdit* startVertexEdit;

    QLabel* resultLabel;
    QLabel* stepLabel;

    QTimer* animationTimer;
    int currentStep;

    void setupUI();
    void setupInitialGraph();

    // Для анимации BFS
    std::vector<int> bfsQueue;
```

```

std::vector<int> bfsVisited;
int bfsCurrentVertex;

// Для анимации DFS
std::vector<int> dfsStack;
std::vector<int> dfsVisited;
int dfsCurrentVertex;

// Для анимации Дейкстры
std::vector<int> dijkstraVisited;
int dijkstraCurrentVertex;
std::map<int, double> dijkstraDist;

public:
    MainWindow(QWidget* parent = nullptr);
    ~MainWindow();

private slots:
    void addVertex();
    void removeVertex();
    void addEdge();
    void removeEdge();

    void startBFS();
    void animateBFSStep();

    void startDFS();
    void animateDFSStep();

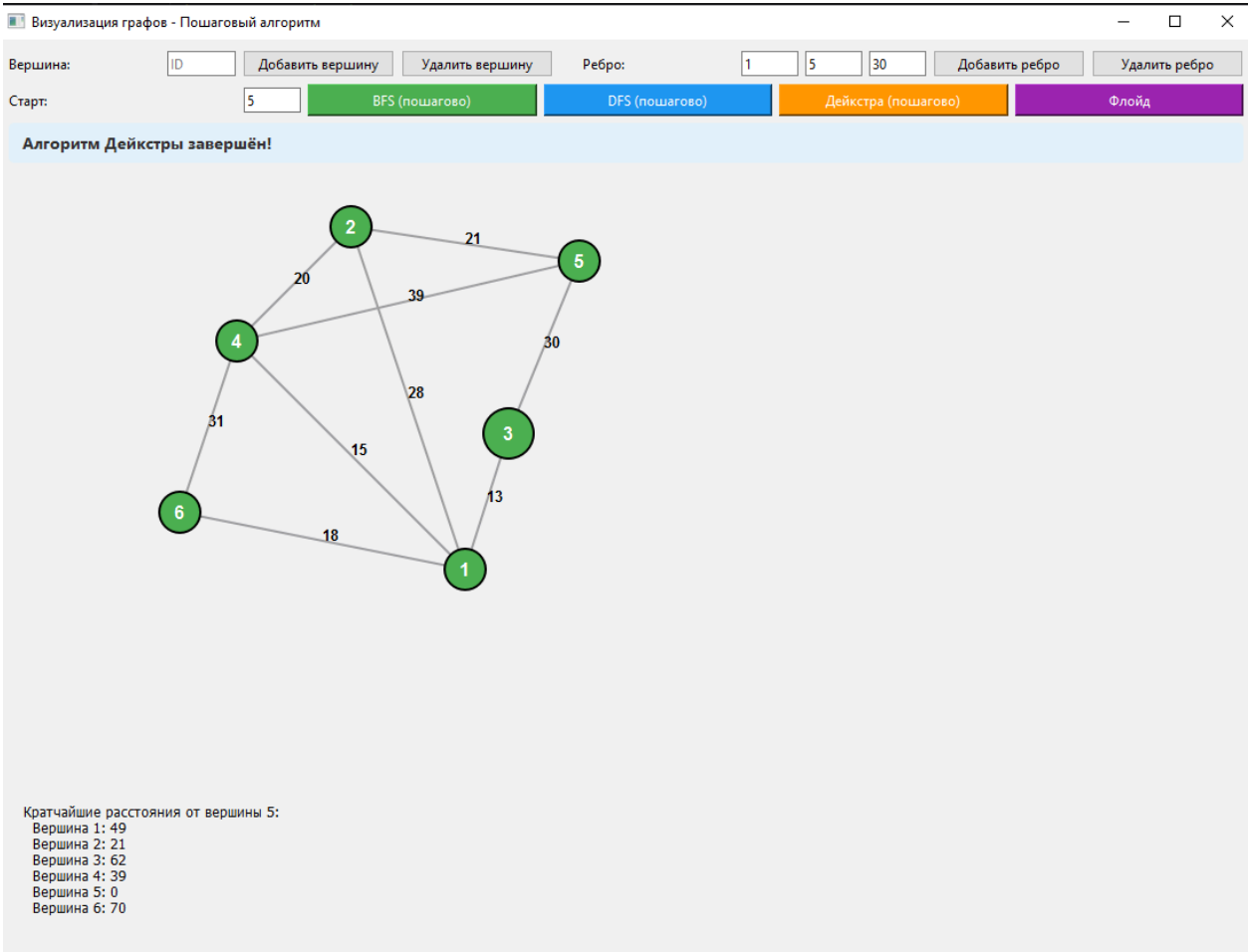
```

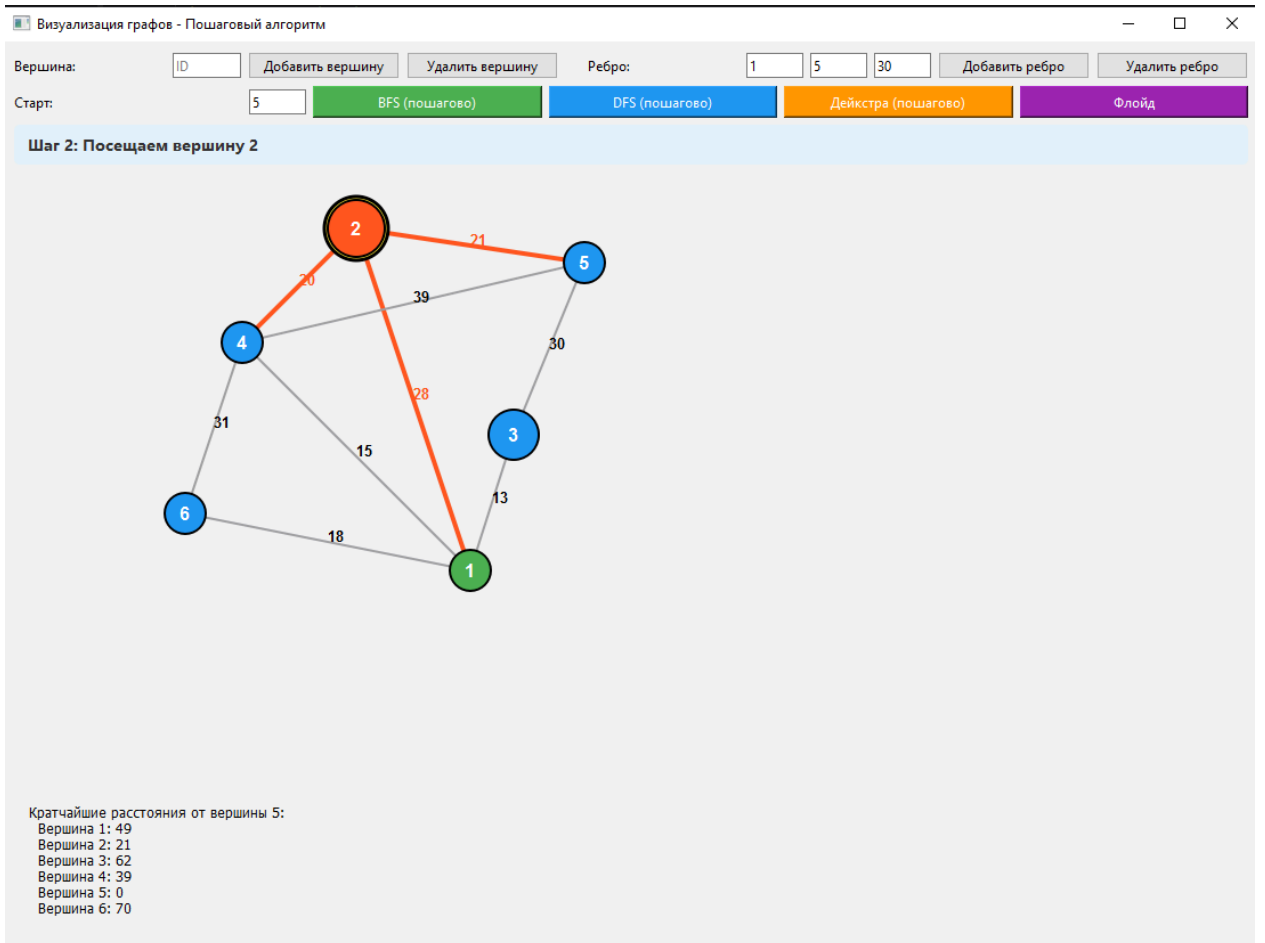
```
void startDijkstra();  
void animateDijkstraStep();  
  
void runFloyd();  
};  
  
#endif // MAINWINDOW_H
```

main.cpp

```
#include <QApplication>  
#include "mainwindow.h"  
  
int main(int argc, char *argv[]) {  
    QApplication a(argc, argv);  
    MainWindow w;  
    w.show();  
    return a.exec();  
}
```

Результаты работы:





UML

